ML Toolkit Setup & Utility Module

Machine Learning Environment and Reusable Python Utilities

Vijay Kota

1. Introduction

A proper machine learning workflow begins with a correctly configured development environment. Before working with datasets or building machine learning models, it is important to understand how Python environments, packages, modules, and reusable functions work together.

The objective of this assignment is to set up an isolated Python environment, install and verify commonly used machine learning libraries, and develop a small reusable utility module.

The assignment demonstrates the use of:

- Python virtual environments
- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- Python modules
- Reusable functions
- Importing functionality between Python files

No machine learning model is trained in this assignment. The primary focus is on environment setup, modular programming, and reusable tools for future machine learning workflows.

2. Objectives

The main objectives of this assignment are:

1. Create an isolated Python virtual environment.
2. Install NumPy, Pandas, Matplotlib, and Scikit-learn.
3. Verify that all required libraries are installed correctly.
4. Create a reusable Python utility module.
5. Implement at least three functions useful for machine learning and data-analysis workflows.
6. Import the utility module into a separate Python script.

7. Execute and verify the complete workflow.
8. Organize and document the project properly.

3. Development Environment

The project was developed using Python and an isolated virtual environment.

3.1 Python Version

The virtual environment was created using:

Python 3.9

The virtual environment was created using:

```
python3 -m venv .venv
```

The environment was activated using:

```
source .venv/bin/activate
```

Using a virtual environment ensures that project dependencies are isolated from the system-level Python installation.

4. Required Libraries

The following libraries were installed for the project:

Library	**Version**	**Purpose**
NumPy	2.0.2	Numerical computing and array operations
Pandas	2.3.3	Data manipulation and analysis
Matplotlib	3.9.4	Data visualization
Scikit-learn	1.6.1	Machine learning utilities and algorithms

The libraries were installed using:

```
pip install numpy pandas matplotlib scikit-learn
```

The Python package manager was also upgraded before installation:

```
pip install --upgrade pip
```

5. Installation Verification

After installation, each required library was imported successfully and its installed version was displayed.

The verification command used was:

```
python -c "import numpy as np; import pandas as pd; import matplotlib; import
    sklearn; print('NumPy:', np.__version__); print('Pandas:', pd.__version__);
    print('Matplotlib:', matplotlib.__version__); print('Scikit-learn:', sklearn.
    __version__); print('All ML libraries imported successfully!')"
```

The verified versions were:

- NumPy: **2.0.2**
- Pandas: **2.3.3**
- Matplotlib: **3.9.4**
- Scikit-learn: **1.6.1**

The successful import of all four libraries confirms that the required machine learning toolkit was correctly installed.

6. Project Structure

The project was organized as follows:

```
ML_Toolkit_Setup/
|
|-- .venv/
|       |-- Python virtual environment
|
|-- ml_utils.py
|       |-- Reusable ML utility functions
|
|-- main.py
|       |-- Main execution script
|
|-- README.md
|       |-- Project documentation
```

The `.venv` directory contains the isolated Python environment and its installed packages.

The `ml_utils.py` file contains reusable functions.

The `main.py` file imports and uses those functions.

The `README.md` file provides project documentation and usage instructions.

7. Reusable Utility Module

A reusable Python module named:

was created.

The module contains five reusable functions:

1. `dataset_summary(df)`
2. `missing_value_report(df)`
3. `remove_missing_rows(df)`
4. `normalize_array(values)`
5. `detect_outliers_iqr(series)`

These functions were designed to perform common tasks that may be useful during exploratory data analysis and future machine learning workflows.

7.1 `dataset_summary(df)`

The `dataset_summary()` function generates a basic summary of a Pandas DataFrame.

It returns:

- Number of rows
- Number of columns
- Total number of missing values
- Number of duplicate rows

Example usage:

```
summary = dataset_summary(df)
```

This function is reusable for quickly understanding the basic structure and quality of a dataset.

7.2 `missing_value_report(df)`

The `missing_value_report()` function calculates the number and percentage of missing values for every column.

Example usage:

```
report = missing_value_report(df)
```

The function returns a Pandas DataFrame containing:

- Missing value count
- Missing value percentage

This is useful during exploratory data analysis because missing values often require preprocessing before machine learning.

7.3 remove_missing_rows(df)

The `remove_missing_rows()` function removes rows containing missing values and returns a new copy of the DataFrame.

Example usage:

```
cleaned_df = remove_missing_rows(df)
```

Returning a copy prevents unintended modification of the original DataFrame.

7.4 normalize_array(values)

The `normalize_array()` function performs Min-Max normalization using NumPy.

The normalization formula is:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

The resulting values are generally scaled between 0 and 1.

Example:

```
values = np.array([10, 20, 30, 40, 50])

normalized = normalize_array(values)
```

The result is:

$$[0,\ 0.25,\ 0.5,\ 0.75,\ 1]$$

Normalization can be useful when numerical features have substantially different scales.

7.5 detect_outliers_iqr(series)

The `detect_outliers_iqr()` function identifies potential statistical outliers using the Interquartile Range method.

The IQR is defined as:

$$IQR = Q_3 - Q_1$$

The lower and upper boundaries are:

$$Lower = Q_1 - 1.5(IQR)$$

$$Upper = Q_3 + 1.5(IQR)$$

Values outside these boundaries are identified as potential outliers.

Example usage:

```
outliers = detect_outliers_iqr(df["income"])
```

The function returns a Boolean mask that can be used to locate the outlier observations.

8. Main Script and Module Import

A separate file named:

`main.py`

was created to demonstrate the use of the utility module.

The functions are imported using:

```
from ml_utils import (
    dataset_summary,
    missing_value_report,
    remove_missing_rows,
    normalize_array,
    detect_outliers_iqr
)
```

This demonstrates that the utility functions are stored separately from the main program and can be reused through Python's import mechanism.

9. Testing

A small sample DataFrame was created in `main.py` to test the utility functions.

The sample data contains variables representing:

- Age
- Salary
- Department

The utility functions were then executed against the sample data.

The testing included:

1. Generating a dataset summary.
2. Generating a missing-value report.
3. Removing missing rows.
4. Normalizing numerical values.
5. Detecting statistical outliers.

10. Execution and Results

The main script was executed using:

```
source .venv/bin/activate
python main.py
```

The script executed successfully without errors.

The final output confirmed:

```
UTILITY MODULE EXECUTED SUCCESSFULLY
```

This demonstrates that:

- The virtual environment works correctly.
- Required libraries can be imported.
- The utility module can be imported.
- Functions from the utility module execute successfully.
- The separate main script can use the reusable functions.

11. Reusability of the Utility Module

The main advantage of creating `ml_utils.py` is that its functions can be reused in different machine learning projects.

For example, another project can simply import the required function:

```
from ml_utils import normalize_array
```

and then use:

```
scaled_values = normalize_array(values)
```

This avoids rewriting the same functionality in every project.

The module therefore acts as a small personal toolkit for common data science and machine learning operations.

12. Code Organization and Documentation

The project follows a modular structure.

The responsibilities are separated as follows:

File	Responsibility
`ml_utils.py`	Reusable utility functions
`main.py`	Testing and execution of utilities
`README.md`	Project documentation and setup instructions
`.venv/`	Isolated Python environment

Each utility function contains documentation explaining its purpose, parameters, and return value.

This makes the module easier to understand and reuse.

13. Assignment Requirement Mapping

The implementation satisfies the assignment requirements as follows:

Requirement	**Implementation**	**Marks**
Environment setup and verification	Virtual environment created and four required libraries installed and verified	2
Reusable utility functions	Five reusable functions implemented in `ml_utils.py`	4
Import and usage from separate script	`main.py` imports and executes functions from `ml_utils.py`	3
Code organization and documentation	Modular project structure, docstrings, and README	1
Total		**10**

14. Screenshot Evidence

Two screenshots should be included as evidence of the implementation.

14.1 Library Installation Verification

The first screenshot should show the terminal command used to import all required libraries and display their installed versions.

The expected output contains:

```
NumPy: 2.0.2
Pandas: 2.3.3
Matplotlib: 3.9.4
Scikit-learn: 1.6.1
All ML libraries imported successfully!
```

Screenshot 1: Terminal showing successful library verification.

14.2 Utility Module Execution

The second screenshot should show the execution of:

```
python main.py
```

The terminal should show the utility functions executing and the final successful execution message.

Screenshot 2: Terminal showing successful execution of `main.py` and the reusable utility functions.

15. Conclusion

This assignment successfully established a Python environment for machine learning development and demonstrated the use of reusable Python modules.

A dedicated virtual environment was created using Python 3.9, and NumPy, Pandas, Matplotlib, and Scikit-learn were successfully installed and verified.

A reusable utility module named `ml_utils.py` was developed with five functions for common machine learning and data-analysis tasks: dataset summarization, missing-value analysis, missing-row removal, normalization, and IQR-based outlier detection.

A separate `main.py` script successfully imported and used these functions, demonstrating Python's modular import system.

The project establishes a foundation for future machine learning assignments by encouraging good environment management, modular code organization, reusable functionality, and proper documentation.

— End of Report —